AVALIAÇÃO DE DESEMPENHO ENTRE ARQUITETURAS MONOLÍTICA E DE MICROSSERVIÇOS

Eduardo Vinicius Reimer[1], Leanderson André[2]

[1]Bacharelado em Engenharia de Sofware
[2]Universidade da Região de Joinville (Univille) Joinville – SC – Brazil

eduardo.reimer@univille.br, leandersonandre@univille.br

Resumo. *A disponibilidade, o desempenho e a estabilidade são requisitos essenciais para assegurar o bom funcionamento de softwares, especialmente durante períodos de alta concorrência de usuários simultâneos. Diferentes padrões de arquitetura apresentam vantagens e desvantagens que impactam diretamente o desempenho. O objetivo deste estudo é avaliar o desempenho entre sistemas monolíticos e uma arquitetura baseada em microsserviços, por meio da realização de testes de carga. Para isso, foram desenvolvidas duas aplicações ASP.NET Core 8.0: uma com arquitetura monolítica e outra com microsserviços. Nela foram realizados testes de carga utilizando o Grafana K6. Foram avaliadas métricas de quantidade total de requisições, percentual de erros, tempo médio de resposta, latência e tráfego de rede. A análise estatística pelo teste de Wilcoxon mostrou que a arquitetura monolítica teve melhor desempenho sob cargas moderadas, enquanto a de microsserviços apresentou maior estabilidade sob alta demanda.*

Palavras-chave: *Arquitetura monolítica, Arquitetura de microsserviços, Desempenho, Testes de carga.*

Abstract. *Availability, performance, and stability are critical requirements for ensuring robust software operation, particularly under high simultaneous user loads. Various architectural patterns offer distinct advantages and disadvantages that directly influence performance. This study aims to compare the performance of a monolithic architecture against a microservices-based architecture by conducting load tests. For this purpose, two applications were developed using ASP.NET Core 8.0: one monolithic and one based on microservices. These applications were subjected to load testing with Grafana k6. Metrics such as total requests, error percentage, average response time, latency, and network throughput were evaluated. Statistical analysis via the Wilcoxon test revealed that the monolithic architecture performed better under moderate loads, whereas the microservices architecture exhibited greater stability under high demand.*

Keywords: *Monolithic architecture, Microsservices architecture, Performance, Load testing.*

1. Introdução

A arquitetura de um software é um elemento fundamental para o entendimento e o desenvolvimento de um sistema, representando aquilo que desenvolvedores experientes identificam como essencial, o que torna o seu conceito profundamente contextual e

coletivo. Em vez de ser uma lista de decisões fixas, a arquitetura é um acordo implícito sobre o que realmente importa no design do software (Fowler, 2003). Uma arquitetura mal planejada ou deficiente contribui para o surgimento de obstáculos que dificultam a compreensão completa do software, tornando o desenvolvimento de novas funcionalidades mais lento e propenso a erros (Fowler, 2019).

Embora a definição de arquitetura não seja consensual na área, ela pode ser compreendida por meio de sua estrutura, decisões e princípios de design. A estrutura do sistema define o estilo de implementação, geralmente dividido entre monolítico e distribuído (Ford e Richards, 2024). Um sistema monolítico é uma unidade de implantação onde todas as funcionalidades do sistema devem ser implantadas em conjunto, enquanto a arquitetura de microsserviços, um exemplo de sistema distribuído, consiste em serviços que podem ser implantados de maneira independente e são modelados em torno do domínio de negócio (Newman, 2020).

Na medida que as aplicações crescem em tamanho, torna-se mais difícil mantê-las e realizar modificações. Embora o software monolítico ainda possa ser mantido e desenvolvido, eventualmente torna-se evidente que mudanças na arquitetura de toda a aplicação precisam ser feitas. Essa tendência foi observada inicialmente em empresas com alto volume de tráfego, grande número de desenvolvedores e bases de código extensas. Organizações como Amazon, Netflix, LinkedIn e SoundCloud migraram para uma arquitetura de microsserviços porque suas aplicações monolíticas se tornaram difíceis de manter, desenvolver e escalar (Kalske, Mäkitalo, Mikkonen, 2018).

Durante a migração de sistemas monolíticos para uma arquitetura de microsserviços, é fundamental ter um profundo conhecimento do negócio para evitar a definição incorreta das fronteiras dos serviços (Newman, 2020). Além disso, desafios relacionados à comunicação entre os serviços, à orquestração dos mesmos em produção e a questões técnicas de infraestrutura podem se tornar fatores críticos no processo de migração (Kalske, Mäkitalo, Mikkonen, 2018).

Diante desse cenário, este artigo teve como objetivo avaliar o desempenho entre sistemas monolíticos e arquitetura baseada em microsserviços, por meio da realização de testes de carga. Para isso, foram desenvolvidas duas aplicações com o mesmo escopo funcional: uma utilizando arquitetura monolítica e outra baseada em microsserviços. A questão de pesquisa que norteia este trabalho é: como a migração de uma arquitetura monolítica para microsserviços impacta o desempenho?

A metodologia adotada inclui a definição de cenários de teste, seleção de métricas de desempenho relevantes e a execução de um experimento controlado. A análise dos resultados abrange a comparação entre as arquiteturas com base nas métricas obtidas, seguida de uma discussão crítica sobre os achados. Por fim, a conclusão destaca qual arquitetura apresentou melhor desempenho segundo os critérios definidos.

2. Revisão da literatura

Nesta etapa, foram discutidos os padrões de arquitetura com ênfase na arquitetura de microsserviços e monolítica, destacando suas principais características. Além disso, são discutidos os testes de desempenho, com ênfase nos testes de carga, empregados para avaliar a performance das arquiteturas mencionadas.

2.1. Sistemas distribuídos

Uma arquitetura de sistema distribuído possui diversos serviços autônomos e que irão se apresentar ao usuário como um sistema único e coerente. Portanto, o fato de se

apresentar de maneira coesa para o usuário, que desconhece a distribuição por trás, torna-se uma característica desse tipo de estrutura (Tanenbaum e Steen, 2007). Além disso, devido à existência de componentes independentes, a facilidade de aumentar ou expandir a escala dos sistemas se destaca ao analisar sistemas desse tipo (Tanenbaum e Steen, 2007).

Entretanto, mesmo que apresente pontos fortes relacionados à escalabilidade, disponibilidade e desempenho, sistemas distribuídos possuem algumas desvantagens, como a confiabilidade da rede, custo do transporte, segurança da rede, a potencial latência, manutenção e versionamento do contrato (Ford e Richards, 2024).

2.1.1. Arquitetura monolítica

Quando todas as funcionalidades de um sistema tiverem de ser implantadas em conjunto, esse sistema será considerado monolítico (Newman, 2020). Conforme essas aplicações têm a necessidade de expansão, expandem também a dificuldade e o risco em dar manutenção ao código existente e adicionar novas funcionalidades, visto que o sistema é interdependente (Kalske, Mäkitalo, Mikkonen, 2018).

Mesmo com as desvantagens existentes em um monolito, essa abordagem como estrutura inicial traz benefícios por permitir explorar a complexidade do sistema e limitações de seus componentes (Fowler, 2015).

2.1.2. Arquitetura de microsserviços

Microsserviços são serviços que podem ser implantados de forma independente e são modelados em torno de um domínio de negócio. Eles se comunicam entre si por meio de redes e, como uma opção de arquitetura, oferecem diversos modos de resolver os problemas que você poderá enfrentar. Isso implica que uma arquitetura de microsserviços tem como base vários microsserviços que colaboram entre si (Newman, 2020).

Para Fowler (2015), a arquitetura de microsserviços pode ser útil em certos casos, pois ela provê vantagens como: contexto definido, que é importante para grandes times atuarem; implantações independentes, que permitem implantar serviços mais facilmente e com menos probabilidade de apresentarem problemas; e a diversidade tecnológica, que permite aos desenvolvedores utilizarem diversas linguagens, frameworks e bancos de dados devido ao baixo acoplamento. Também há desvantagens, como: sistemas distribuídos são mais difíceis de programar, chamadas remotas de APIs terão tempo de resposta mais alto e haverá margem para o risco de elas falharem.

2.2. Teste de desempenho

O teste de desempenho de software é essencial para garantir a qualidade e eficiência do sistema, mas apresenta desafios, principalmente devido à falta de clareza nas definições e classificações, como destaca Souza (2018). O autor observa que, apesar da existência de normas como a ISO/IEC/IEEE 29119-2, os testes de desempenho exigem adaptações específicas para identificar gargalos e ajustar o sistema de acordo com suas necessidades. O teste de desempenho não apenas avalia a eficiência do software, mas também busca identificar limitações que possam comprometer seu funcionamento, como falhas em consultas, sistemas operacionais ou servidores (Souza, 2018).

Também há cinco tipos de classificações de desempenho: carga, estresse, capacidade, resistência e volume, sendo fundamental a aplicação de métodos específicos para cada tipo de teste a fim de garantir um sistema eficiente e de alta qualidade (Souza, 2018). Além disso, o autor enfatiza a importância de um entendimento claro dos requisitos

de desempenho e de como documentá-los adequadamente, alertando que a terminologia inconsistente pode gerar mal-entendidos entre os profissionais envolvidos (Souza, 2018).

2.2.1. Teste de carga

Teste de carga é um tipo de teste de desempenho que mede a capacidade de um sistema, como um site ou aplicação web, de atender a um volume de acessos e interações previsto para sua operação normal. Esse teste simula o comportamento de múltiplos usuários acessando o sistema simultaneamente, replicando suas ações reais por meio de scripts que reproduzem interações comuns de clientes. Menascé (2002) afirma que a ideia é avaliar o desempenho da aplicação em termos de fatores críticos, como tempo de resposta e taxa de transferência (throughput), especialmente quando o sistema está sob alta demanda. Para isso, é utilizado um gerador de carga, que emula navegadores virtuais (usuários virtuais), enviando solicitações ao sistema e mensurando as respostas, o que ajuda a verificar a capacidade de suportar o tráfego esperado.

Os resultados dos testes de carga permitem identificar pontos fracos na infraestrutura, como gargalos no servidor de banco de dados ou no balanceador de carga, e entender a necessidade de melhorar a capacidade do sistema para garantir que ele permaneça estável e responsivo mesmo em picos de acesso (Menascé, 2002).

2.3. Teste Estatístico

Testes estatísticos são métodos utilizados para verificar se diferenças observadas entre conjuntos de dados são significativas ou se ocorreram por acaso. No contexto da análise de desempenho de sistemas, esses testes permitem validar comparações com base em métricas coletadas de forma empírica (Conover, 1999).

O teste de Wilcoxon para amostras pareadas é uma técnica não paramétrica indicada quando os dados não seguem uma distribuição normal. Ele é utilizado para comparar duas condições aplicadas sobre os mesmos elementos, como dois ambientes de software submetidos ao mesmo teste de carga (Siegel e Castellan, 1988).

O teste calcula as diferenças entre os pares de valores, ordena essas diferenças por magnitude e atribui postos (ranks). Depois, soma separadamente os postos das diferenças positivas e negativas. A estatística do teste (W) é o menor desses somatórios:

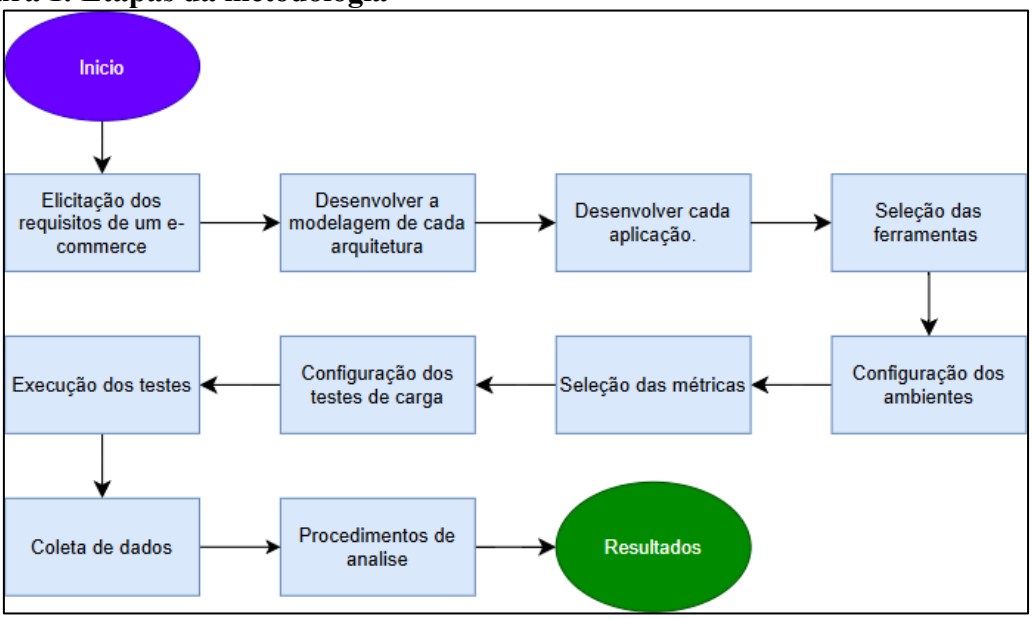
$$W = \min(\sum R^+, \sum R^-)$$

O valor de W é utilizado para calcular o valor de p, que indica se a diferença observada é estatisticamente significativa. Esse teste é especialmente útil quando o tamanho da amostra é pequeno ou os dados contêm outliers, sendo uma alternativa robusta ao teste t pareado (Sheskin, 2004).

3. Procedimentos metodológicos

O trabalho foi desenvolvido seguindo um procedimento metodológico de avaliação comparativa com resultados quantitativos, delineado como uma pesquisa experimental. A metodologia deste estudo foi inspirada no trabalho de (Cabane e Farias, 2022), que propõe a avaliação da performance entre diferentes padrões de arquitetura através de testes de carga. Os testes de carga foram realizados utilizando o Grafana K6, medindo métricas como tempo de resposta, latência, taxa de transferência e número de pacotes recebidos e enviados. Na Figura 1, detalham-se as etapas da metodologia:

Figura 1. Etapas da metodologia



Fonte: Primária (2025)

A seguir serão descritos em tópicos cada etapa da metodologia:

3.1. Elicitação dos requisitos de um E-Commerce

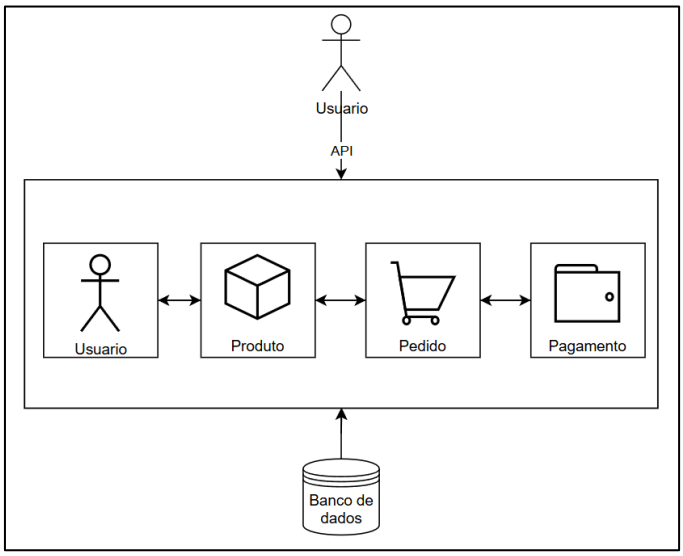
A elicitação de requisitos foi feita com base na identificação das funcionalidades essenciais para o funcionamento da plataforma, considerando a experiência do usuário final e as necessidades de gestão. Foram definidos requisitos como cadastro de produtos, gerenciamento de usuários, carrinho de compras, pedidos e integração com métodos de pagamento. Também foram levados em conta aspectos não funcionais, como desempenho e integridade das informações. A definição dos requisitos seguiu uma abordagem prática, focada na construção de um sistema funcional, claro e alinhado aos objetivos do artigo.

3.2. Desenvolver a modelagem de cada arquitetura

O desenvolvimento da modelagem das arquiteturas foi realizado com base na proposta de implementar a mesma aplicação de e-commerce sob duas abordagens distintas: arquitetura monolítica e arquitetura de microsserviços.

Na Figura 2 é demonstrada a arquitetura monolítica, onde toda a lógica da aplicação foi centralizada em um único projeto, integrando todas as funcionalidades, como cadastro de produtos, gerenciamento de usuários, carrinho, pedidos e pagamentos em um único código-base.

Figura 2. Arquitetura monolítica

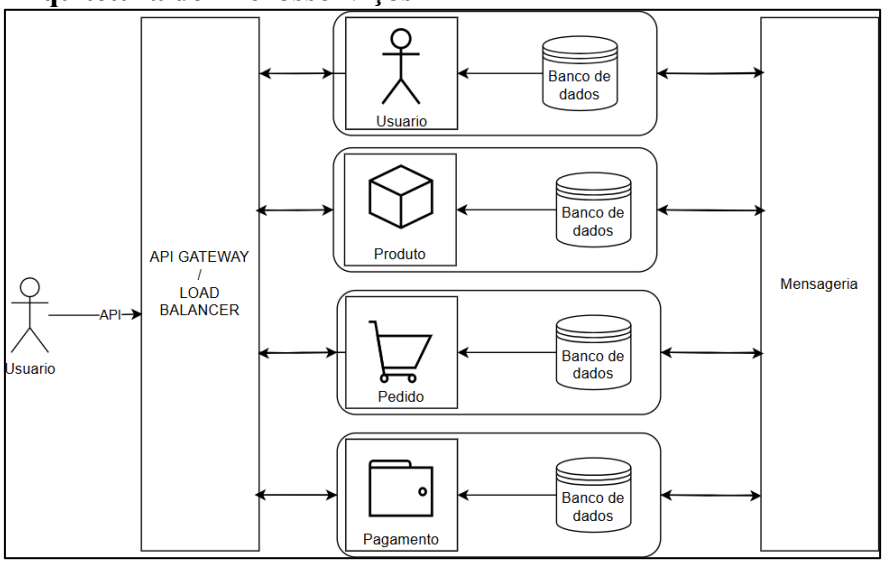


Fonte: Primária (2025)

A modelagem seguiu uma estrutura com módulos organizados por camadas (controladores, serviços, repositórios), e o banco de dados foi compartilhado entre todas as funcionalidades. Esse modelo buscou representar a simplicidade e coesão características de aplicações monolíticas.

Na Figura 3, é demonstrada a arquitetura de microsserviços. O sistema foi dividido em serviços independentes, cada um responsável por um contexto específico, como "Catálogo de Produtos", "Usuários", "Pedidos" e "Pagamentos".

Figura 3. Arquitetura de microsserviços



Fonte: Primária (2025)

Cada serviço foi modelado como uma aplicação separada, com seu próprio banco de dados e API, comunicando-se entre si por meio de chamadas HTTP ou mensageria. A modelagem seguiu os princípios de separação de responsabilidades e baixo acoplamento, visando garantir maior escalabilidade e flexibilidade.

Ambas as arquiteturas foram modeladas utilizando diagramas de entidade-relacionamento, fluxos de dados e mapeamentos de endpoints, o que permitiu uma

visualização clara da estrutura e do comportamento de cada sistema. A escolha por comparar essas duas abordagens teve como objetivo analisar, na prática, as vantagens e desafios de cada modelo no contexto de um e-commerce.

3.3. Desenvolver cada aplicação

Para comparar diferentes arquiteturas, foram desenvolvidos dois projetos de e-commerce com as mesmas regras de negócio, ambos disponíveis publicamente no GitHub. As aplicações foram implementadas em C# com ASP.NET Core 8.0, utilizando SQL Server como banco de dados e executadas em containers Docker.

Na arquitetura monolítica, todas as funcionalidades estão concentradas em uma única aplicação com banco de dados compartilhado. Já na arquitetura de microsserviços, foram criados quatro serviços independentes (produto, usuário, pagamento e pedido), cada um com seu próprio banco de dados. A comunicação entre os serviços ocorre por meio de RabbitMQ e APIs, com consumidores dedicados. Também foi utilizado o Kong API Gateway para roteamento e balanceamento das requisições.

3.4. Seleção das ferramentas

Para a criação e execução dos testes de carga, foi utilizada a ferramenta Grafana K6. É uma ferramenta open-source de testes de carga e performance que simula múltiplos usuários acessando sistemas simultaneamente (Grafana K6, 2025). Usando scripts em JavaScript, permite identificar gargalos e medir métricas como tempo de resposta, latência e taxa de erro. Integrado ao Grafana, fornece visualizações em tempo real para ajudar a garantir a estabilidade e eficiência de aplicações web e APIs.

3.5. Configuração dos ambientes

Os testes foram realizados em um ambiente padronizado, com o objetivo de garantir resultados comparáveis e uma análise precisa. Todos os experimentos ocorreram no mesmo ambiente de teste, mantendo as condições e configurações ao longo das execuções.

As aplicações foram executadas localmente em contêineres utilizando o Docker, uma ferramenta que permite criar e gerenciar ambientes isolados, assegurando consistência e portabilidade na execução das aplicações. A máquina utilizada para os testes era equipada com um processador Intel(R) Core (TM) i5-13420H com 2.10GHz e 16GB de RAM, executando o sistema operacional Windows 11 Pro. Com o intuito de simular um cenário próximo ao de ambientes de nuvem com recursos limitados, como os oferecidos por plataformas como Microsoft Azure e Amazon Web Services (AWS), cada aplicação foi restrita ao uso de no máximo 2 virtual CPUs e 1 GB de memória RAM.

No caso da arquitetura baseada em microsserviços, foram instanciadas três réplicas do serviço de pedidos, duas réplicas do serviço de produtos, uma réplica do serviço do usuário, uma réplica do serviço de pagamento e uma réplica para consumir as mensagens e realizar as ações necessárias, a fim de representar uma distribuição realista de carga e refletir práticas comuns em ambientes escaláveis e resilientes.

3.6. Seleção das métricas

Esta etapa teve como objetivo selecionar as métricas que permitem uma análise detalhada de performance de cada aplicativo alvo. No Quadro 1**,** estão as métricas e suas respectivas definições utilizadas para avaliação dos testes de carga.

Quadro 1. Métricas de performance

Métricas	Definição
Quantidade de requisição total	É a quantidade total de requisições que o script de teste conseguiu realizar.
Percentual de erro	Porcentagem de requisições que resultaram em falhas em relação ao total de requisições executadas durante o teste de carga.
Tempo de resposta médio	É a média do tempo total que um sistema leva para responder a uma requisição. Inclui o tempo de processamento da solicitação e o tempo de envio da resposta de volta ao cliente.
Pacotes recebidos	Indica a quantidade de pacotes de dados recebidos pelo cliente (K6) do servidor.
Pacotes enviados	Representa a quantidade de pacotes de dados enviados pelo cliente (K6) ao servidor.

Fonte: Grafana K6 (2025)

Foram selecionadas as métricas básicas fornecidas pelo Grafana K6, quantidade total de requisição total, percentual de erro, tempo de resposta médio, pacotes recebidos e pacotes enviados.

3.7. Configuração dos testes de carga

Considerando que tanto a aplicação monolítica quanto a baseada em microsserviços implementam as mesmas funcionalidades essenciais de um sistema de comércio eletrônico, foram definidos dois cenários principais para a realização dos testes de carga. O primeiro cenário consiste na execução de requisições ao endpoint responsável pela listagem de todos os produtos. O segundo cenário simula o envio de requisições do tipo POST ao endpoint de pedidos, o qual envolve interações com múltiplos contextos da aplicação, refletindo uma carga computacional mais complexa.

Para cada arquitetura, foram realizadas 30 execuções dos testes. Cada execução simulou acessos simultâneos variando de forma progressiva ao longo de 5 minutos: no primeiro minuto, o número de usuários crescia de 0 a 50; no segundo, de 50 a 100; no terceiro, de 100 a 125; no quarto, havia uma redução de 125 para 100 usuários; e, finalmente, no quinto minuto, o número de usuários diminuía gradualmente de 100 até 0. Essa variação controlada permite observar como as aplicações se comportam diante de diferentes intensidades de carga, incluindo cenários de escalonamento e redução de usuários.

A fim de garantir a repetibilidade e a estabilidade do ambiente de testes, foi desenvolvido um script em PowerShell responsável por automatizar todo o processo. Esse script executava o teste de carga construído com o Grafana K6, realizava a limpeza do banco de dados após cada execução e reconstituía os índices das chaves primárias. Além disso, a cada cinco execuções, o contêiner da aplicação em teste era reiniciado, e a cada dez execuções, o banco de dados também passava por um reinício completo. Essa abordagem foi aplicada de forma idêntica para ambas as arquiteturas, assegurando a equidade nas condições de teste.

3.8. Execução dos testes:

Nesta etapa, foram executados os cenários de teste previamente definidos para cada uma das arquiteturas analisadas, com o objetivo de avaliar seu desempenho sob diferentes condições de carga. A execução foi conduzida de forma automatizada,

utilizando o script PowerShell desenvolvido na fase de preparação, o qual integrou todas as etapas necessárias para garantir a repetibilidade dos testes. Esse script era responsável por iniciar o teste de carga com a ferramenta Grafana K6, realizar a limpeza do banco de dados ao final de cada execução, reinicializar os contêineres da aplicação e garantir que os dados fossem armazenados de forma organizada.

Ao final de cada execução, o Grafana K6 gerava relatórios em formato HTML contendo as métricas de desempenho configuradas, como tempo médio de resposta, taxa de sucesso das requisições, tempo de latência e throughput.

3.9. Coleta dos dados

Esta etapa teve como objetivo principal a coleta e organização dos dados obtidos durante a execução dos cenários de teste realizados com o Grafana K6. Durante cada teste de carga, foram registradas automaticamente diversas métricas previamente definidas, as quais ficaram armazenadas em arquivos no formato HTML.

Após a execução dos testes, foi necessário extrair manualmente esses dados dos arquivos e transferi-los para uma planilha do Excel. Nessa planilha, os dados foram organizados de forma estruturada, sendo separados por arquitetura e agrupados por tipo de métrica e por cenário de teste executado. Essa organização permitiu uma visualização mais clara das informações e facilitou a análise comparativa entre os diferentes testes. Com base nesses dados organizados, foi possível gerar gráficos do tipo boxplot.

3.10. Procedimento de análise

Para comparar o desempenho entre as arquiteturas monolítica e de microsserviços, foi adotado o teste estatístico de Wilcoxon para amostras pareadas. Essa escolha se justifica pela natureza dos dados analisados, que não apresentaram garantia de normalidade, além de envolverem medições emparelhadas sob as mesmas condições de teste, como recomendado por Conover (1999) e Siegel e Castellan (1988).

O teste foi aplicado às métricas coletadas nos cenários de teste repetidos, abrangendo: quantidade total de requisições, percentual de erro, tempo de resposta médio, número de pacotes recebidos e número de pacotes enviados. Para cada par de valores correspondentes às duas arquiteturas, o teste avaliou se as diferenças observadas poderiam ser atribuídas ao acaso ou se eram estatisticamente significativas.

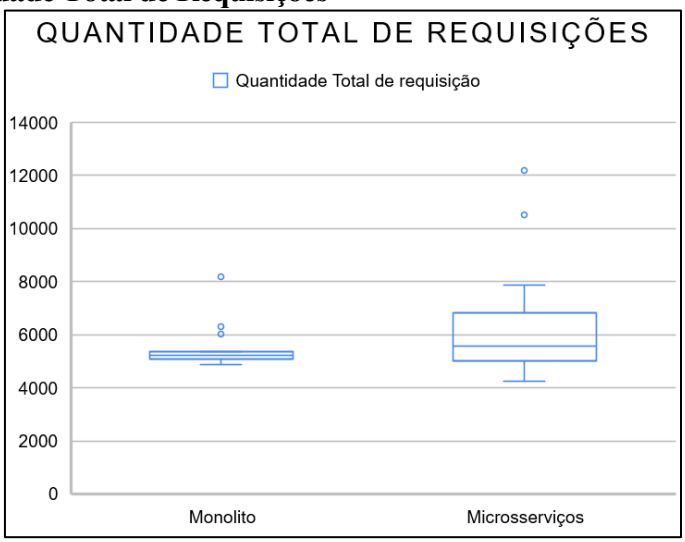
A interpretação dos resultados baseou-se no valor de p (p-valor), que representa a probabilidade de ocorrência de uma diferença igual ou mais extrema que a observada, caso a hipótese nula (ausência de diferença) seja verdadeira. Para fins desta análise, foi adotado um nível de significância de $\alpha = 0,05$, valor amplamente utilizado em pesquisas empíricas por oferecer um bom equilíbrio entre sensibilidade e controle de erro tipo I (Sheskin, 2004). Resultados com p inferior a 0,05 foram considerados estatisticamente significativos, indicando evidência suficiente para rejeitar a hipótese nula.

4. Análise dos dados e discussão dos resultados

A análise estatística dos resultados obtidos, revela que a arquitetura baseada em microsserviços demonstrou vantagens significativas nas métricas, tempo de resposta médio e volume de requisições atendidas. Por outro lado, a arquitetura monolítica apresentou desempenho superior quanto ao percentual de erros. Ao aplicar os testes de Wilcoxon individualmente a cada métrica, permitiu identificar diferenças estatisticamente significativas entre as abordagens.

Na Figura 4, apresenta-se um gráfico boxplot que compara a quantidade total de requisições atendidas pelas diferentes arquiteturas.

Figura 4. Quantidade Total de Requisições

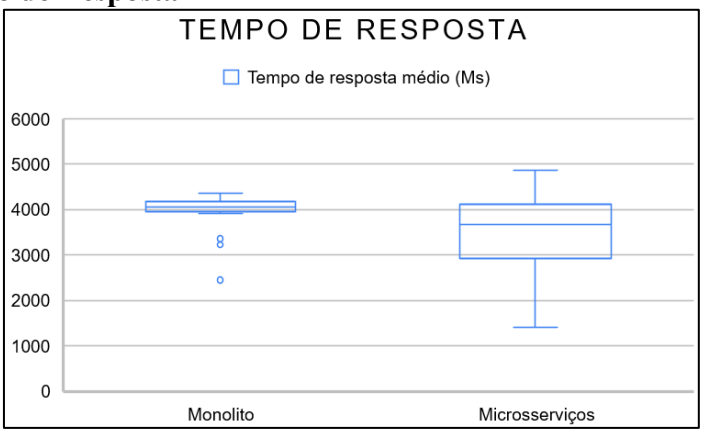


Fonte: Primária (2025)

O teste de Wilcoxon revelou uma diferença estatisticamente significativa entre as arquiteturas $p = 0{,}02202$, com a abordagem baseada em microsserviços demonstrando um desempenho superior no processamento de altos volumes de requisições. Essa superioridade está relacionada à sua escalabilidade, que permite a replicação independente de componentes e uma distribuição de carga mais eficiente.

Na Figura 5, apresenta-se um gráfico boxplot que compara os tempos de resposta médios observados nas diferentes arquiteturas.

Figura 5. Tempo de Resposta

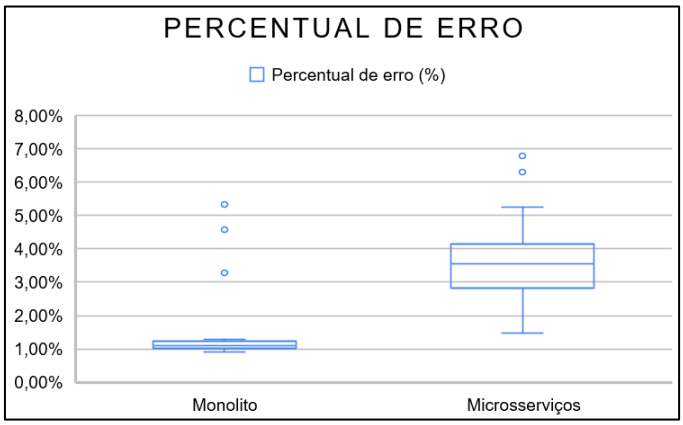


Fonte: Primária (2025)

O teste de Wilcoxon apresentou valor de $p = 0{,}00466$, indicando diferença estatisticamente significativa entre as arquiteturas. A abordagem de microsserviços obteve tempos de resposta médios menores que a monolítica, o que sugere maior eficiência no processamento das requisições, possivelmente devido à segmentação e otimização de serviços específicos.

Na Figura 6, apresenta-se um gráfico boxplot que compara o percentual de erros registrados durante a execução dos testes para as diferentes arquiteturas.

Figura 6. Percentual de Erro

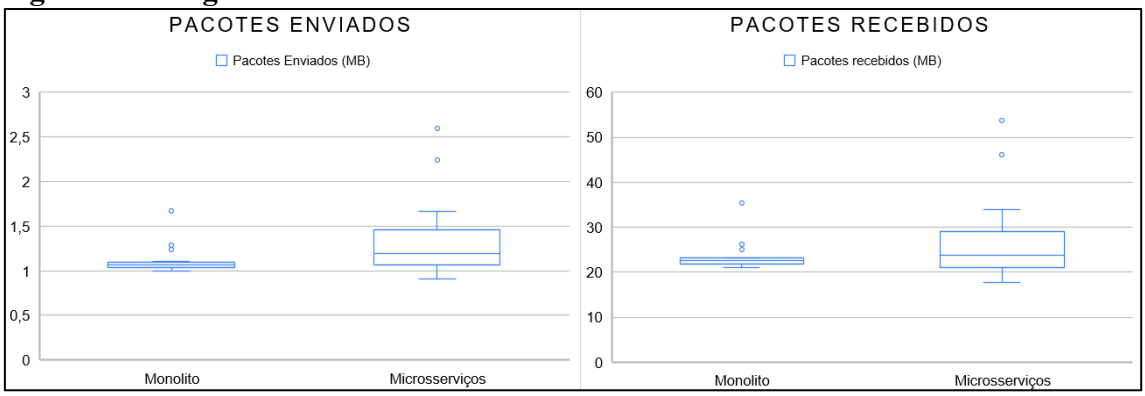


Fonte: Primária (2025)

A análise estatística pelo teste de Wilcoxon resultou em um valor de $p = 0{,}00001$, indicando uma diferença altamente significativa. Observou-se que a arquitetura monolítica apresentou percentuais de erro consistentemente menores. Embora os microsserviços tenham mostrado maior capacidade de requisições, eles também apresentaram taxas de erro mais altas, sugerindo menor robustez operacional. Isso pode ser atribuído à complexidade da comunicação entre serviços distribuídos e à presença de componentes intermediários, como o RabbitMQ, que aumentam os pontos de falha e demandam estratégias mais sofisticadas de tolerância a falhas e consistência para garantir a estabilidade do sistema.

Na Figura 7, apresenta-se um gráfico boxplot que compara o volume de tráfego de rede, medido em pacotes enviados e recebidos, entre as arquiteturas avaliadas.

Figura 7. Tráfego de Rede



Fonte: Primária (2025)

Os testes estatísticos indicaram diferenças significativas em ambas as métricas, com valores-p igual a 0,02320 para pacotes recebidos e igual a 0,00374 para pacotes enviados. Em ambos os casos, a arquitetura de microsserviços apresentou volumes superiores. Esse aumento no tráfego é consistente com o modelo distribuído dos microsserviços, que requer múltiplas interações entre componentes. Como resultado, há um custo adicional de comunicação, exigindo uma infraestrutura de rede mais robusta para garantir o desempenho esperado.

A arquitetura monolítica apresentou um comportamento mais estável e previsível sob carga, evidenciado por suas menores taxas de erro e consumo reduzido de rede. Essa eficiência operacional, resultante de sua baixa dependência de comunicação entre

componentes, sugere que essa abordagem pode ter um impacto favorável nos custos operacionais de infraestrutura.

Por outro lado, a análise de desempenho deve ser considerada em conjunto com suas implicações nos custos. Embora a arquitetura de microsserviços demonstre vantagens em escalabilidade e tempos de resposta, sua estrutura distribuída inerentemente eleva a complexidade. Essa complexidade, por sua vez, pode refletir em um aumento dos custos gerais, devido à necessidade de mais recursos, orquestração e comunicação entre os serviços.

A escolha entre uma arquitetura monolítica ou baseada em microsserviços, portanto, depende das necessidades específicas de cada sistema. É crucial entender que o impacto financeiro não se limita aos recursos de computação diretos. Fatores como a complexidade de gerenciar múltiplos serviços, o monitoramento distribuído e o ecossistema de ferramentas de suporte também influenciam o custo total de propriedade, sendo considerações importantes em cenários onde a simplicidade e o controle de custos são relevantes.

5. Conclusão

O presente estudo teve como objetivo avaliar o desempenho entre sistemas monolíticos para uma arquitetura baseada em microsserviços, por meio da realização de testes de carga. Para isso, foi conduzido um experimento empírico envolvendo dois sistemas de e-commerce implementados com essas respectivas arquiteturas, os quais foram submetidos a testes de carga com base em um conjunto definido de métricas de desempenho.

A quantidade total de requisições, percentual de erro, tempo de resposta médio, pacotes recebidos e pacotes enviados foram avaliados durante os testes para observar o desempenho das duas arquiteturas sob condições controladas de estresse. Os resultados indicaram que a arquitetura de microsserviços apresentou desempenho superior em termos de volume de requisições processadas, tempos de resposta mais baixos e maior tráfego de rede, refletido nos pacotes transmitidos e recebidos. Além dessas vantagens técnicas, destaca-se a flexibilidade operacional proporcionada por essa arquitetura, como a possibilidade de escalabilidade horizontal e a facilitação do trabalho de equipes independentes em diferentes partes do sistema. No entanto, esses benefícios trazem consigo alguns desafios, como a necessidade de decompor o banco de dados monolítico, gerenciar a orquestração entre os serviços e lidar com o aumento no consumo de recursos computacionais.

Dessa forma, os resultados obtidos oferecem subsídios para desenvolvedores e arquitetos de software que precisam escolher a arquitetura mais adequada às necessidades de seus projetos.

Para trabalhos futuros, é possível adotar uma abordagem mais completa, considerando escalar as aplicações na nuvem, como na AWS ou no Azure, pode ser uma estratégia viável para colocar custos operacionais na comparação e obter dados mais robustos e detalhados.

Referências

CABANE, Hebert; KLEINNER FARIAS. On the impact of event-driven architecture on performance: An exploratory study. Future Generation Computer Systems, v. 153, p. 52–69, 2022.

FORD, M.; RICHARDS, N. Fundamentos da arquitetura de software: uma abordagem de engenharia. Rio de Janeiro: Alta Books, 2024.

FOWLER, M. Who needs an architect? IEEE Software, v. 20, n. 5, p. 11–13, 2003.

FOWLER, M. Monolith First. 2015. Disponível em: https://martinfowler.com/bliki/MonolithFirst.html. Acesso em: 15 mar. 2025.

FOWLER, M. Software Architecture Guide. 2019. Disponível em: https://martinfowler.com/architecture/. Acesso em: 20 mar. 2025.

GRAFANA LABS. End of test summary. 2025. Disponível em: https://grafana.com/docs/k6/latest/results-output/end-of-test/. Acesso em: 12 abr. 2025.

KALSKE, M.; MÄKITALO, N.; MIKKONEN, T. Challenges when moving from monolith to microservice architecture. Manuscrito, 2018.

MENASCÉ, D. A. Load testing of web sites. IEEE Internet Computing, v. 6, n. 4, p. 70–74, 2002.

NEWMAN, S. Migrando sistemas monolíticos para microsserviços. Sebastopol: O'Reilly Media, 2020.

SOUZA, T. Testes de Desempenho de Software: Teoria e Prática. Porto Alegre: Sociedade Brasileira de Computação, 2018.

TANENBAUM, A. S.; STEEN, M. V. Sistemas distribuídos: princípios e paradigmas. São Paulo: Pearson/Prentice Hall, 2007.

CONOVER, W. J. Practical Nonparametric Statistics. 3. ed. New York: Wiley, 1999.

SIEGEL, Sidney; CASTELLAN Jr., N. John. Nonparametric Statistics for the Behavioral Sciences. 2. ed. New York: McGraw-Hill, 1988.

SHESKIN, David J. Handbook of Parametric and Nonparametric Statistical Procedures. 3. ed. Boca Raton: Chapman & Hall/CRC, 2004.